PROJET DÉVELOPPEMENT MOBILE ANDROID

Kotlin · Jetpack Compose · MVVM · Room · API REST · Cartographie

Application de Supervision NanoOrbit Ground Control

CubeSat Earth Observation System — NanoOrbit

Niveau 2ème année cycle ingénieur	Groupes Binômes ou trinômes	Module ALTN82 — Développement Mobile Android

SGBD miroir	Ce projet est le pendant mobile du projet NanoOrbit ALTN83 (Oracle 23ai). Les modèles de données et règles métier sont volontairement cohérents entre les deux modules.

1. Contexte et présentation du projet

La startup NanoOrbit exploite une constellation de CubeSats pour surveiller des zones climatiques sensibles : déforestation, fonte des glaces, qualité de l'air et évolution du trait de côte. Les ingénieurs de terrain et les opérateurs des centres de contrôle ont besoin d'une application mobile native pour superviser la constellation en temps réel, même lorsqu'ils se trouvent en déplacement sur un site de lancement ou dans un avion.

Ce projet constitue le fil rouge du module ALTN82. Il est organisé en trois phases progressives qui couvrent l'ensemble du programme :

- Phase 1 — Interface & Données : Compose, état, listes, données simulées
- Phase 2 — Architecture MVVM & API REST : ViewModel, StateFlow, Retrofit
- Phase 3 — Fonctionnalités Avancées : Navigation multi-écrans, Room, Géolocalisation, Carte

1.1 Périmètre fonctionnel

L'application NanoOrbit Ground Control couvre quatre domaines fonctionnels :

- La supervision de la flotte : visualisation des satellites, statuts, orbites
- La consultation des fiches détail : télémétrie, instruments embarqués, historique
- Le planning des communications : fenêtres de passage par station au sol
- La cartographie : visualisation géographique des stations au sol

1.2 Synergie ALTN82 / ALTN83

Bien que les deux projets soient évalués séparément, la cohérence des données est primordiale. Trois points de synergie obligatoires sont attendus et évalués :

Point de synergie	Attendu côté Android	Lien avec ALTN83
Modèles de données	La data class Kotlin Satellite doit correspondre aux colonnes de la table SATELLITE Oracle (types, noms, contraintes)	Table SATELLITE du MLD de référence — section 2 du sujet ALTN83
Règles métier	Si le trigger Oracle bloque une fenêtre > 900 s, l'app Android doit afficher une erreur de validation identique côté client	RG-F04 (durée fenêtre), Trigger T1 — Phase 2 ALTN83
Disponibilité réseau	L'application doit expliquer et implémenter sa stratégie hors-ligne (cache Room, messages d'erreur)	Question Q3 de la Phase 1 ALTN83 : continuité de service en cas d'indisponibilité du serveur central

PHASE 1 Interface & Données Jetpack Compose · État · LazyColumn · Données simulées			
Durée indicative	3 h 30	**Prérequis**	Android Studio configuré — TP1 terminé

1.1 Objectifs pédagogiques

- Créer une application Android structurée avec Jetpack Compose
- Définir des data classes Kotlin cohérentes avec le MLD NanoOrbit
- Implémenter des composants réutilisables (SatelliteCard, WindowCard)
- Gérer l'état local avec remember { mutableStateOf() } et la recomposition
- Afficher des listes performantes avec LazyColumn sur des données simulées

1.2 Étape 1 — Modèles de données Kotlin

Créez un fichier Models.kt contenant les data classes qui reflètent le MLD de référence ALTN83. Chaque data class doit être documentée avec les correspondances Oracle.

Data class Kotlin	Table Oracle miroir	Champs obligatoires (non-null)	Champs optionnels (nullable)
Satellite	SATELLITE	idSatellite, nomSatellite, statut, formatCubesat, idOrbite	dateLancement, masse
Orbite	ORBITE	idOrbite, typeOrbite, altitude, inclinaison	zoneCouverture
Instrument	INSTRUMENT	refInstrument, typeInstrument, modele	resolution, consommation
FenetreCom	FENETRE_COM	idFenetre, datetimeDebut, duree, statut, idSatellite, codeStation	volumeDonnees
StationSol	STATION_SOL	codeStation, nomStation, latitude, longitude	diametreAntenne, debitMax
Mission	MISSION	idMission, nomMission, objectif, dateDebut, statutMission	dateFin, zoneGeoCible

ℹ Cohérence des types : le statut d'un satellite (Opérationnel / En veille / Défaillant / Désorbité) doit utiliser exactement les mêmes valeurs que le CHECK Oracle. Déclarez-le comme une enum class Kotlin pour garantir la compatibilité.

1.3 Étape 2 — Jeu de données simulées (Mock Data)

Créez un fichier MockData.kt contenant au minimum les données suivantes, correspondant au jeu de données de référence ALTN83 :

- 5 satellites dont 1 Désorbité (SAT-001 à SAT-005)
- 3 orbites (2 SSO, 1 LEO) avec leurs paramètres
- 4 instruments avec leurs caractéristiques
- 5 fenêtres de communication (3 Réalisées, 2 Planifiées)
- 3 stations au sol avec latitude/longitude réelles

⚠ Point de vigilance : le satellite SAT-005 de statut Désorbité doit être présent dans vos données simulées. L'application doit le traiter différemment (pastille grise, interaction désactivée). Ce comportement miroir le trigger T1 Oracle.

1.4 Étape 3 — Composants Compose

Implémentez les composants réutilisables suivants. Chaque composant doit être testé avec @Preview.

Composant	Paramètres	Comportement attendu
SatelliteCard	satellite: Satellite, onClick: () -> Unit	Pastille colorée selon statut (vert/orange/rouge/gris), nom, format CubeSat, orbite
StatusBadge	statut: StatutSatellite, modifier: Modifier	Chip Material3 colorée : Opérationnel=vert, En veille=orange, Défaillant=rouge, Désorbité=gris
FenetreCard	fenetre: FenetreCom, nomStation: String	Durée formatée, badge statut (Planifiée/Réalisée), volume si disponible
InstrumentItem	instrument: Instrument, etatFonctionnement: String	Type, modèle, résolution (N/A si null), indicateur état

1.5 Étape 4 — Écran Dashboard (DashboardScreen)

Implémentez l'écran principal affichant la liste de tous les satellites. Cet écran constitue le point d'entrée de l'application.

- LazyColumn des satellites avec SatelliteCard pour chaque entrée
- Barre de recherche TextField avec filtrage temps réel par nom ou type d'orbite
- Compteur de satellites en tête de liste (ex : "8/10 satellites opérationnels")
- Gestion visuelle des satellites désorbités (carte grisée, texte DÉSORBITÉ)
- Message d'état affiché pendant le chargement

1.6 Questions de réflexion (à répondre en commentaire dans le code)

Réf.	Question
Q1	Pourquoi utilise-t-on LazyColumn plutôt que Column pour la liste des satellites ? Quel problème de performance Column poserait-il avec 100 satellites ?
Q2	La pastille de couleur StatusBadge dépend du statut. Pourquoi une enum class Kotlin est-elle préférable à une String libre pour ce champ ?
Q3	Le satellite désorbité (SAT-005) ne devrait plus avoir de nouvelles fenêtres de communication. Comment l'application peut-elle empêcher l'utilisateur de planifier une fenêtre pour ce satellite ? Comparez avec le trigger Oracle T1.

1.7 Livrables Phase 1

Réf.	Livrable	Contenu attendu
L1-A	**Models.kt**	Toutes les data classes avec commentaires de correspondance Oracle et enum class StatutSatellite
L1-B	**MockData.kt**	Jeu de données cohérent avec ALTN83 (5 satellites, 3 orbites, 4 instruments, 5 fenêtres, 3 stations)
L1-C	**Composants @Composable**	SatelliteCard, StatusBadge, FenetreCard, InstrumentItem avec @Preview fonctionnelles
L1-D	**DashboardScreen**	LazyColumn avec recherche, compteur, gestion désorbités, réponses Q1–Q3 en commentaire

PHASE 2 Architecture MVVM & API REST ViewModel · StateFlow · Retrofit · Gestion d'état			
Durée indicative	3 h 00	**Prérequis**	Phase 1 terminée — TPs 2 et 3 réalisés

2.1 Objectifs pédagogiques

- Restructurer l'application selon l'architecture MVVM (Model - View - ViewModel)
- Séparer les préoccupations : Models.kt, VeloViewModel, écrans Compose
- Exposer l'état via StateFlow et l'observer avec collectAsStateWithLifecycle
- Consommer un service REST simulé ou réel avec Retrofit + Gson
- Gérer les états de chargement, de succès et d'erreur dans l'interface

2.2 Étape 1 — Restructuration MVVM

Réorganisez le projet en trois couches distinctes. Chaque couche a une responsabilité unique :

Fichier / Couche	Rôle	Règle d'or
Models.kt — Modèle	Data classes, enums, interface API Retrofit	Aucune dépendance Android. Pur Kotlin. Testable unitairement.
NanoOrbitViewModel.kt — ViewModel	États (MutableStateFlow), logique réseau, filtrage, actions utilisateur	Ne connaît pas l'interface. Survit aux rotations. Utilise viewModelScope.
***Screen.kt — Vue**	Composables Compose uniquement. Aucun appel réseau direct.	Observe le ViewModel via collectAsStateWithLifecycle. Délègue les événements.

2.3 Étape 2 — NanoOrbitViewModel

Créez NanoOrbitViewModel.kt héritant de ViewModel(). Ce ViewModel centralise tout l'état de l'application.

États à exposer via StateFlow :

- satellites : StateFlow<List<Satellite>> — liste complète des satellites
- isLoading : StateFlow<Boolean> — indicateur de chargement
- errorMessage : StateFlow<String?> — message d'erreur (null si aucune erreur)
- searchQuery : StateFlow<String> — texte de recherche en cours
- selectedStatut : StateFlow<StatutSatellite?> — filtre actif (null = tous)

Fonctions publiques à implémenter :

- loadSatellites() — déclenche le chargement (viewModelScope.launch)
- onSearchQueryChange(query: String) — met à jour searchQuery
- onStatutFilterChange(statut: StatutSatellite?) — met à jour le filtre
- refreshSatellites() — recharge les données depuis la source

> ℹ Calcul côté ViewModel : la liste filtrée (satellites filtrés par searchQuery et selectedStatut) doit être calculée dans le ViewModel via combine() ou en tant que StateFlow dérivé, pas dans le composable. Cela garantit la testabilité de la logique de filtrage.

2.4 Étape 3 — Couche données (Repository + API)

Implémentez un Repository qui abstrait la source de données. L'API simulée doit respecter la structure JSON du MLD NanoOrbit :

- Créez NanoOrbitRepository.kt avec une méthode suspend getSatellites(): List<Satellite>
- Implémentez NanoOrbitApi (interface Retrofit) avec les endpoints GET /satellites, GET /satellites/{id}/instruments, GET /fenetres
- Simulez une latence réseau (delay(500)) pour tester les états de chargement
- Gérez les exceptions réseau dans le ViewModel (try/catch + mise à jour de errorMessage)

> ⚠ La durée d'une fenêtre de communication est bornée à [1, 900] secondes par la règle RG-F04 (trigger Oracle T3). L'application doit valider cette contrainte côté client avant tout envoi de données. Affichez un message d'erreur explicite si la valeur saisie dépasse 900 secondes.

2.5 Étape 4 — Connexion Vue ↔ ViewModel

Mettez à jour DashboardScreen pour consommer les StateFlow du ViewModel :

- Injectez le ViewModel via viewModel() (paramètre par défaut dans la signature du composable)
- Observez chaque StateFlow avec collectAsStateWithLifecycle()
- Affichez un CircularProgressIndicator pendant isLoading == true
- Affichez un message d'erreur avec bouton Réessayer si errorMessage != null
- Branchez TextField sur onSearchQueryChange du ViewModel

2.6 Étape 5 — Filtres par statut

Ajoutez une barre de chips de filtrage permettant de n'afficher que les satellites d'un statut donné :

- FilterChip pour chaque valeur de StatutSatellite + un chip Tous
- Le chip actif est surligné (selected = true)
- Le filtre s'applique en combinaison avec la recherche textuelle
- Un compteur "{n} résultat(s)" est mis à jour en temps réel

2.7 Livrables Phase 2

Réf.	Livrable	Contenu attendu
L2-A	**NanoOrbitViewModel.kt**	6 StateFlow exposés, 4 fonctions publiques, gestion viewModelScope et try/catch
L2-B	**NanoOrbitRepository.kt + NanoOrbitApi.kt**	Interface Retrofit GET /satellites + /instruments, simulation de latence, gestion d'exception
L2-C	**DashboardScreen mis à jour**	Connecté au ViewModel, états loading/error/success, filtres statut fonctionnels
L2-D	**Validation RG-F04**	Contrôle client durée [1–900 s] avec message d'erreur lisible, comparaison avec trigger Oracle en commentaire

PHASE 3 Fonctionnalités Avancées Navigation · Room · Géolocalisation · Cartographie · Notifications			
Durée indicative	4 h 00	**Prérequis**	Phases 1 et 2 terminées — TPs 3 et 4 réalisés

3.1 Objectifs pédagogiques

- Implémenter une navigation multi-écrans avec Navigation Compose (NavHost, routes)
- Créer un écran de détail satellite avec télémétrie et instruments embarqués
- Ajouter la persistance locale avec Room pour le mode hors-ligne
- Intégrer la géolocalisation GPS pour la proximité des stations
- Afficher une carte OpenStreetMap (osmdroid) avec les stations au sol
- (Bonus) Implémenter des notifications locales pour les passages imminents

3.2 Étape 1 — Navigation Compose

Structurez la navigation de l'application autour d'un NavHost central. Définissez les routes dans un objet Routes.

Route	Écran	Paramètres URL	Accès
dashboard	DashboardScreen — liste satellites	—	Onglet bas + démarrage
detail/{satelliteId}	DetailScreen — fiche satellite	satelliteId: String	Clic carte satellite
planning	PlanningScreen — fenêtres com.	—	Onglet bas
map	MapScreen — stations sol sur carte	—	Onglet bas

Implémentez une BottomNavigationBar avec les onglets Dashboard, Planning et Carte. La barre ne doit pas être visible sur DetailScreen.

3.3 Étape 2 — DetailScreen (Fiche Satellite)

Cet écran affiche l'ensemble des informations d'un satellite. Il reçoit le satelliteId en paramètre de route et récupère les données depuis le ViewModel.

- En-tête : TopAppBar avec nom du satellite et bouton Retour (ArrowBack)
- Section Statut : StatusBadge, format CubeSat, type d'orbite, altitude
- Section Télémétrie : masse, capacité batterie (avec indicateur visuel), durée de vie restante estimée
- Section Instruments embarqués : LazyColumn des InstrumentItem avec état de fonctionnement
- Section Missions : liste des missions actives avec rôle du satellite
- Bouton Signaler une anomalie : ouvre un dialog de saisie (texte libre + validation)

3.4 Étape 3 — PlanningScreen (Fenêtres de communication)

Cet écran affiche le planning chronologique des fenêtres de communication :

- Sélecteur de station (DropdownMenu ou FilterChip) parmi les stations disponibles
- Liste triée par datetime_debut des FenetreCard
- Indicateur de durée totale de contact et volume total planifié
- Distinction visuelle claire Planifiée (bleu) / Réalisée (vert) / Annulée (rouge)
- Validation côté client : durée [1–900 s], satellite non désorbité (miroir RG-S06)

3.5 Étape 4 — Persistance locale avec Room

Implémentez un cache Room pour permettre à l'application de fonctionner en mode hors-ligne :

Entité Room	Table SQLite générée	Données cachées
SatelliteEntity	satellites	Tous les champs de la data class Satellite (dernière mise à jour)
FenetreEntity	fenetres_com	Fenêtres des 7 prochains jours

- Le Repository doit implémenter une stratégie Cache-First : lecture locale en premier, puis mise à jour réseau en arrière-plan
- Une bannière "Mode hors-ligne" doit apparaître si les données proviennent du cache
- L'âge des données cachées doit être affiché (ex : "Mis à jour il y a 42 min")

> ℹ Lien ALTN83 Q3 : la stratégie Cache-First répond directement à la question Q3 de la Phase 1 ALTN83 ("Comment Singapour peut-il continuer à planifier si le serveur central est indisponible ?"). Commentez explicitement ce lien dans NanoOrbitRepository.kt.

3.6 Étape 5 — MapScreen (Cartographie OSM)

Intégrez osmdroid pour afficher les stations au sol sur une carte OpenStreetMap :

- Marqueurs colorés par état : Opérationnelle (vert), En maintenance (orange), Hors service (gris)
- Infobulle avec nom, bande de fréquence et débit max au clic sur un marqueur
- Bouton FAB "Me localiser" pour centrer la carte sur la position GPS de l'opérateur
- Distance à chaque station affichée dans la bulle si la géolocalisation est active

3.7 Bonus — Notifications locales

Ce bonus est optionnel et valorisé dans la notation (voir palier Excellence) :

- Notification locale 15 minutes avant le début d'une fenêtre de communication Planifiée
- Utilisation de WorkManager avec PeriodicWorkRequest (vérification toutes les 5 minutes)
- Contenu de la notification : nom du satellite, station, durée du passage
- Canal de notification dédié avec priorité PRIORITY_HIGH

3.8 Livrables Phase 3

Réf.	Livrable	Contenu attendu
L3-A	**Routes.kt + Navigation**	NavHost avec 4 routes, BottomNavigationBar 3 onglets, paramètres corrects
L3-B	**DetailScreen**	5 sections présentes, dialog anomalie, TopAppBar avec retour fonctionnel
L3-C	**PlanningScreen**	Sélecteur station, tri chronologique, validation RG-F04/S06, badges couleur
L3-D	**Room + Repository**	2 entités, stratégie Cache-First, bannière hors-ligne, lien ALTN83 Q3 commenté
L3-E	**MapScreen**	Carte OSM, marqueurs colorés par état, bulle info, bouton Me localiser
L3-F ★	**Notifications (Bonus)**	WorkManager, notification 15 min avant passage, canal dédié

4. Récapitulatif de l'évaluation

Phase	Contenu	Thèmes ALTN82 couverts	Points
Phase 1	Interface & Données	Compose, State, LazyColumn, data classes, @Preview	20
Phase 2	Architecture MVVM & API REST	ViewModel, StateFlow, Retrofit, Repository, filtres	25
Phase 3	Fonctionnalités Avancées	Navigation, Room, GPS, osmdroid, BottomNav	25
Bonus ★	Notifications + Pull-to-Refresh + Favoris	WorkManager, DataStore, PullToRefreshBox	+10
TOTAL			**70**

5. Paliers de réalisation

Pour faciliter la progression, les fonctionnalités sont organisées en trois paliers. Chaque palier est indépendant et évaluable séparément.

Palier	Composants Android mobilisés	Attendu
Socle (Obligatoire)	@Composable, LazyColumn, remember/mutableStateOf, ViewModel, StateFlow, collectAsStateWithLifecycle, Navigation Compose (NavHost, NavController), Retrofit	Phases 1 et 2 complètes + navigation de base + DetailScreen
Avancé	Room (Entity, DAO, Database), Cache-First Repository, PlanningScreen avec validation, MapScreen osmdroid, BottomNavigationBar	Phase 3 complète (hors bonus). Architecture 3 couches propre.
Excellence ★	WorkManager (PeriodicWorkRequest, CoroutineWorker), DataStore Preferences, PullToRefreshBox Material3, Glance AppWidget (optionnel avancé)	Toutes les phases + au moins 2 bonus sur 3 (Notifications, Pull-to-Refresh, Favoris)

6. Organisation et modalités de rendu

Modalité	Détail
Groupes	Binômes ou trinômes — constitués librement en début de module
Technologie cible	Android (API 26+), Kotlin, Jetpack Compose, Android Studio Ladybug ou supérieur
Rendu	Archive ZIP contenant le projet Android Studio complet (sans dossier build/) + un README.md de 1 page décrivant les choix d'implémentation et les bonnes pratiques retenues
Nom du fichier	GROUPE_NomA_NomB_NanoOrbit_Android.zip
Présentation orale	15 min de démonstration live sur émulateur ou téléphone réel + 10 min de questions techniques — l'application doit se lancer sans erreur au moment de la présentation
Cohérence ALTN83	Le README doit explicitement mentionner les 3 points de synergie (modèles, RG-F04, hors-ligne Q3) et indiquer comment ils sont implémentés côté Android
Plagiat	Tout partage de code entre groupes est sanctionné par la note 0/20 et une convocation en conseil de discipline

Annexe — Correspondance MLD NanoOrbit ↔ Data Classes Kotlin

Ce tableau de correspondance est fourni à titre indicatif. Les étudiants sont libres d'adapter les noms en camelCase Kotlin standard, sous réserve de documenter les correspondances en commentaire.

Colonne Oracle (MLD)	Type Oracle	Champ Kotlin	Type Kotlin	Remarque
id_satellite	VARCHAR2(20)	idSatellite	String	Format SAT-001
nom_satellite	VARCHAR2(100)	nomSatellite	String	
statut	VARCHAR2(30)	statut	StatutSatellite (enum)	Enum class Kotlin
format_cubesat	VARCHAR2(5)	formatCubesat	FormatCubeSat (enum)	1U/3U/6U/12U
date_lancement	DATE	dateLancement	LocalDate?	Nullable
duree (FENETRE_COM)	NUMBER	dureeSecondes	Int	[1–900] RG-F04
datetime_debut	TIMESTAMP	datetimeDebut	LocalDateTime	Formatage HH:mm
latitude (STATION_SOL)	NUMBER(9,6)	latitude	Double	Pour osmdroid

ℹ Enum classes recommandées : StatutSatellite (OPERATIONNEL, EN_VEILLE, DEFAILLANT, DESORBITE), FormatCubeSat (U1, U3, U6, U12), StatutFenetre (PLANIFIEE, REALISEE, ANNULEE), TypeOrbite (SSO, LEO, MEO, GEO). Ces enums garantissent la cohérence avec les contraintes CHECK Oracle.